MODULE 29 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 3 of 3

The last twenty questions, and a look at what three papers have told you.

QUESTIONS 20 HOLD 9 seconds PAPER 3 of 3

admin@sthithapragna.com

> A team is told Terraform's multi-cloud support means their AWS configuration will work against Azure with minimal change.

How should this be corrected?

A It is accurate, provided only official providers are used

B It is accurate once the configuration is written with modules

C It is accurate for compute resources but not storage

D Terraform offers one workflow across providers, not portable configuration

◆ PAUSE AND DECIDE

How should this be corrected?

A **With official providers**

Provider tier has nothing to do with it. Resource types differ between clouds regardless of who maintains the provider.

B **With modules**

A module wraps resource types; it does not translate between them. The resources inside are still cloud-specific.

C **Compute but not storage**

Invents a distinction. No category of resource is portable between clouds.

D **One workflow, not portable config**

Correct. The promise is a common write, plan, apply cycle across providers. Configuration is written against specific resource types.

ON THE EXAM

Domain 1. 'Write once, run on any cloud' is the most common misreading of what multi-cloud means here.

> A bank has written its own provider for an internal system. Security forbids publishing it anywhere outside the bank, and the platform team would rather not operate another internal service.

How should it be distributed?

A Give it a source address on a bank-controlled hostname and use a filesystem mirror

B Publish it to the public registry under a private namespace

C Omit it from required_providers and place the binary manually

D Rewrite it as a module, since providers must be published

◆ PAUSE AND DECIDE

How should it be distributed?

A Own hostname, mirror

Correct. A source address is an identifier first; the hostname need not resolve. A mirror avoids running a registry.

B Private namespace

The public registry has no private namespaces. A namespace is an organisation name, not an access control.

C Omit and place manually

Omitting source implies the hashicorp namespace. Providers must be declared to be used.

D Rewrite as a module

A module composes resource types that already exist. Only a provider can create a new one.

ON THE EXAM

Domain 2. A source address looks like a URL, so people assume it must resolve. It does not.

admin@sthithapragna.com

> Available provider releases are 1.4.0, 1.4.7, 1.5.2 and 2.0.1. A configuration specifies a constraint of ~> 1.4.0.

Which version is selected?

A 1.4.0

B 1.4.7

C 1.5.2

D 2.0.1

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which version is selected?

A 1.4.0

The floor, not the selection. Terraform takes the newest version satisfying the constraint.

B 1.4.7

Correct. Three components written means the rightmost may rise, so patch releases only. 1.5.0 and above are excluded.

C 1.5.2

What ~> 1.4 with two components would select. The third component changes the meaning entirely.

D 2.0.1

What >= 1.4.0 with no ceiling would select. The pessimistic operator always implies a ceiling.

ON THE EXAM

Domain 2. Count the components before anything else. Three allow patch; two allow minor. Same operator, different answer.

> A team wants every engineer and every pipeline to install exactly the same provider versions, without pinning each one to an exact number in configuration.

What achieves this?

A Add .terraform to version control

B Run terraform init -upgrade in the pipeline

C Commit the .terraform.lock.hcl file

D Use a plugin cache directory shared across machines

◆ PAUSE AND DECIDE

What achieves this?

A Commit .terraform

A disposable working directory holding provider binaries. It belongs in your ignore file, not your repository.

B -upgrade in the pipeline

The opposite. It re-resolves to the newest matching versions every run, maximising divergence.

C Commit the lock file

Correct. It records the exact version selected and its checksums, so everyone selects identically.

D A shared plugin cache

Saves downloading. It does not decide which version each configuration selects.

ON THE EXAM

Domain 2. Two filenames differ by four characters and have opposite treatment: ignore the directory, commit the lock file.

> A repository has passed terraform fmt -check -recursive for two years. Nobody has touched the Terraform code this week. After the pipeline image was rebuilt with a newer Terraform CLI, the check now fails on forty files.

What is the most likely explanation?

A The newer CLI applies additional formatting rules, which is expected

B -recursive now scans directories it previously skipped

C A third-party tool reformatted the repository

D fmt -check is non-deterministic and should be replaced

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the most likely explanation?

A New rules in a new CLI

Correct. HashiCorp states the canonical format may change in minor ways between versions, and new rules are not a breaking change.

B -recursive changed

Plausible-sounding and unsupported. Recursive has meant subdirectories since it was introduced.

C A third-party tool

The image rebuild changed the tool, not the repository. Nothing in a pipeline writes back to source.

D Non-deterministic

Entirely deterministic for a given version, which is the whole point of a canonical format.

ON THE EXAM

Domain 3. Inputs constant, output changed, therefore the tool changed. Run fmt once, commit, carry on.

> A pipeline must raise an alert only when a plan finds pending changes, and must treat a genuine error differently from a plan with changes.

What supports this?

A Parse the plan output for the summary line

B terraform plan -json, and count the change entries

C terraform plan -detailed-exitcode

D terraform plan -refresh=false, which exits non-zero on changes

◆ PAUSE AND DECIDE

What supports this?

A **Parse the output**

Works until the wording changes. Terraform provides a supported mechanism, so parsing human output is unnecessary.

B **-json and count**

Would work and is far more effort. It also still needs logic to distinguish an error from a change.

C **-detailed-exitcode**

Correct. Zero means no changes, one means an error, two means changes present - exactly the three-way distinction required.

D **-refresh=false**

Skips reading reality. It has no effect on exit codes at all.

ON THE EXAM

Domain 3. Without this flag a successful plan exits zero whether or not it found changes, which is why pipelines need it.

> A deployment script captures a load balancer hostname with terraform output and passes it to a health check. Every check fails, with the hostname wrapped in double quotation marks.

What should the script use?

A terraform show, filtered to the output name

B terraform output -raw

C terraform console, piped the output name

D terraform state show on the load balancer

◆ PAUSE AND DECIDE

What should the script use?

A show, filtered

show prints whole state or a whole plan file. It does not take an output name.

B output -raw

Correct. -raw prints the bare value with no quoting, which is what a shell variable needs.

C console piped

Console can evaluate the expression, and wrapping an interactive tool in a script is the wrong instrument.

D state show

Prints a resource's recorded attributes for a human to read, not a single value for a script.

ON THE EXAM

Domain 3. The quotes are not a bug. Default output is formatted for a terminal; -raw is formatted for another program.

> A database's generated administrator password is supplied through a variable declared with sensitive = true. Plan output correctly hides it. An auditor opens the remote state file and reads the password in plain text.

What is the explanation?

A sensitive applies to outputs, not variables

B The backend should encrypt sensitive attributes individually

C The plan was run with -refresh=false

D sensitive affects displayed output only. State stores values in plain text

◆ PAUSE AND DECIDE

What is the explanation?

A Outputs only

Both variables and outputs accept sensitive, and it behaves the same way in both. The marking worked as designed.

B Per-attribute encryption

No backend encrypts individual attributes. Encryption at rest protects the whole file from theft, not from authorised readers.

C -refresh=false

Controls whether attributes are re-read. It has no bearing on whether they are stored.

D Display only

Correct. sensitive suppresses display. HashiCorp's guidance is to treat state itself as sensitive data.

ON THE EXAM

Domain 4. This course's most repeated misconception. Nothing malfunctioned; the mental model was wrong.

> A module author writes a dynamic block labelled lifecycle, so callers can switch prevent_destroy on by passing a value. Terraform rejects the configuration.

Why?

A Dynamic blocks cannot generate meta-argument blocks such as lifecycle

B Dynamic blocks require a non-empty for_each

C The label must be quoted and needs an iterator

D prevent_destroy is unavailable inside modules

◆ PAUSE AND DECIDE

Why?

A Cannot generate meta-arguments

Correct. Terraform processes meta-arguments before it is safe to evaluate expressions, so they cannot be produced dynamically.

B Needs non-empty for_each

An empty collection is valid and simply generates no blocks. That is a normal pattern.

C Quoting and iterator

Invents a rule. No keyword becomes valid by adding an iterator argument.

D Not available in modules

prevent_destroy works anywhere a lifecycle block is valid. Module scope is irrelevant.

ON THE EXAM

Domain 4. The same principle governs plan-time for_each keys and literal-only lifecycle values: what the engine needs early cannot come from what runs late.

> A module takes a boolean enable_logging and a string log_bucket_name. If logging is on, the bucket name must not be empty. The team runs Terraform 1.12.

Where should this rule live?

A A null_resource carrying a precondition, the documented workaround

B A check block, which can see both variables

C Nowhere. Terraform cannot express a rule spanning two variables

D A validation block on the variable, which since 1.9 may reference others

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Where should this rule live?

A null_resource precondition

The documented pattern before 1.9. On 1.12 it adds a placeholder resource to state for no benefit.

B A check block

Warns rather than blocking, so an invalid combination would deploy anyway, and it runs last.

C Cannot be expressed

True before 1.9 and false since. This is the belief the question is testing.

D Cross-object validation

Correct. Since 1.9 a validation condition may reference other input variables, data sources and local values.

ON THE EXAM

Domain 4. Option A catches experienced people rather than beginners, because it was the correct answer for several years.

> A team reads database credentials from Vault using the Vault provider's data sources. A security review finds the passwords in plain text in the remote state file. They are on Terraform 1.12 with a current provider.

What should they change?

A Mark the data source output as sensitive

B Enable encryption at rest on the backend

C Replace the data source with an ephemeral resource

D Move the credentials into TF_VAR_ environment variables

◆ PAUSE AND DECIDE

What should they change?

A Mark it sensitive

Display only, as ever. The value would still be written to state.

B Encryption at rest

Worth doing, and it does not resolve this. The reviewer had legitimate read access.

C An ephemeral resource

Correct. Same secret, same source, and the value is never written to the plan artefact or state.

D Environment variables

Moves the secret out of Vault into pipeline configuration, losing central rotation, and it would still land in state.

ON THE EXAM

Domain 4. Fetching from a secrets manager solved distribution and never solved persistence. Ephemeral resources closed that gap.

> A team proposes publishing a module containing exactly one storage bucket resource, with eleven input variables each passed straight through to the corresponding resource argument.

What should a reviewer say?

A Approve. Every resource should be wrapped for consistency

B Probably not. It adds a layer and a release cycle while encapsulating no decision

C Approve once count is added so it can be made conditional

D Reject. The registry does not accept single-resource modules

◆ PAUSE AND DECIDE

What should a reviewer say?

A Wrap everything

Consistency for its own sake, and it restricts callers to the eleven arguments chosen where the resource offers more.

B Encapsulates nothing

Correct. Eleven pass-through variables is a rename, not an abstraction. A module should save the caller from knowing something.

C Add count first

Improves a module that should not exist. Conditional creation works equally well on the resource directly.

D Registry rejects it

Invents a rule. A single-resource module is fine when it encodes an opinion, such as enforced encryption or naming.

ON THE EXAM

Domain 5. A single-resource module is not automatically wrong. Pure pass-through is what makes this one questionable.

> A platform team distributes modules from a private Git repository. Every consuming team needs a deploy key and every pipeline needs repository credentials, so onboarding takes days. They already use HCP Terraform.

What removes most of that burden?

A Make the module repository readable by everyone

B Vendor the modules into each repository with a sync job

C Switch consumers to local path sources

D Publish the modules to the HCP Terraform private registry

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What removes most of that burden?

A Open the repository

Widens visibility to solve a distribution problem, and every pipeline still needs VCS credentials.

B Vendor with a sync job

Automates divergence. Every repository still holds its own copy, now with a script to maintain.

C Local paths

Local paths cannot cross repository boundaries reliably, and they accept no version argument.

D Private registry

Correct. The registry stores the source and serves it via API tokens, so consumers need no access to the module repository at all.

ON THE EXAM

Domain 5. Versioning is the obvious benefit of a private registry. Decoupling consumption from repository permissions is the quieter one.

> An engineer runs terraform state rm on a resource so Terraform stops managing it, and leaves the resource block in the configuration. The next plan proposes to create the resource.

Why?

A The resource is declared with no binding in state, so Terraform plans a create

B state rm destroyed the object, so it must be rebuilt

C The state file needs refreshing before the plan is accurate

D state rm requires a matching removed block to take effect

◆ PAUSE AND DECIDE

Why?

A Declared, no binding

Correct. Terraform compares configuration with state, finds a declaration and no binding, and concludes it must create it.

B It was destroyed

state rm destroys nothing. The object keeps running; Terraform simply forgets it.

C Needs refreshing

Refreshing re-reads objects Terraform knows about. It has no record of this one to re-read.

D Needs a removed block

The block is the declarative alternative to the command, not a companion to it.

ON THE EXAM

Domain 6. Applying here builds a duplicate while the original keeps running, unmanaged. Delete the resource block too.

> During an incident, an engineer increased a database's storage through the console. The change was correct and should be kept, and Terraform should manage it correctly from now on. The configuration still specifies the old size.

What should be done?

A Run terraform apply to reconcile state with reality

B Run apply -refresh-only only. State then matches reality

C Run apply -refresh-only, then update the configuration

D Use a removed block so Terraform stops managing the database

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should be done?

A Just apply

Reverts the change. A normal apply makes reality match configuration, and the configuration still says the old size.

B Refresh-only alone

State matches reality and configuration does not, so the next routine apply shrinks the database back.

C Adopt, then codify

Correct. Adopt the change into state, then write it into configuration so the next normal apply proposes nothing.

D A removed block

Stops Terraform managing the database entirely, throwing away management to avoid editing one attribute.

ON THE EXAM

Domain 6. Three responses to drift: revert, adopt, codify. Adopting alone leaves the next apply to undo it, possibly weeks later.

> An engineer writes an import block using for_each over a map of twelve existing databases, then runs terraform plan with -generate-config-out to produce the resource configuration. Terraform errors before generating anything.

What is the cause?

A for_each on import blocks requires Terraform 1.9 or later

B Configuration generation cannot be used with for_each

C The resources must already exist in state

D Import blocks cannot target more than one resource

◆ PAUSE AND DECIDE

What is the cause?

A Needs 1.9

for_each on import blocks arrived in 1.7, and the exam tests 1.12 regardless. Version is not the constraint.

B Generation and for_each

Correct. The flag cannot be used with import blocks that use for_each, nor with resources using for_each or count.

C Must be in state

Backwards. Generation exists for resources not yet in state; once imported, no generation is needed.

D One resource only

Import blocks handle many at once, which is one of their advantages over the command.

ON THE EXAM

Domain 7. Bulk import and configuration generation are the two headline import features, and they are mutually exclusive.

> An apply fails intermittently with an uninterpretable message from the cloud provider. A TRACE log runs to hundreds of megabytes, dominated by Terraform's internal graph output. The team needs the API request and response for the failing call.

What is the better approach?

A TF_LOG_PROVIDER=TRACE

B TF_LOG=DEBUG, for less noise

C TF_LOG_CORE=TRACE

D apply -parallelism=1, so the log covers one operation at a time

◆ PAUSE AND DECIDE

What is the better approach?

A TF_LOG_PROVIDER=TRACE

Correct. It keeps full detail on the half they care about and removes core's graph output entirely.

B DEBUG instead

Reduces volume by lowering detail, risking the loss of the request and response bodies they specifically need.

C TF_LOG_CORE=TRACE

Exactly the output they are already drowning in. Core produced the noise, not the API calls.

D -parallelism=1

Makes the log easier to follow and does nothing about its size or which half is talking.

ON THE EXAM

Domain 7. Narrow by component before narrowing by level. Lowering the level discards the detail you need.

> A policy requires instances to come from an approved size list. Occasionally a genuine exception is justified, and when one is granted it must be a deliberate decision with a record of who approved it.

Which enforcement level applies?

A Advisory

B Hard-mandatory

C Soft-mandatory

D Hard-mandatory, with a separate advisory policy for exceptions

◆ PAUSE AND DECIDE

Which enforcement level applies?

A Advisory

Logs a warning and lets the run continue, so nothing would be deliberate and no approval would be required.

B Hard-mandatory

Permits no override at all. The requirement explicitly allows justified exceptions.

C Soft-mandatory

Correct. It stops the run, permits an override by someone with the right permission, and the override is recorded.

D Hard plus advisory

Two policies that between them still cannot grant an exception. An advisory policy blocks nothing.

ON THE EXAM

Domain 8. Paper one question 17 was 'never, by anyone', which is hard-mandatory. Here an exception is legitimate, so the level differs.

> A team manages clusters in a private data centre with no route from HCP Terraform's runners. They want to keep policy enforcement, run history and centrally managed variables.

What should they use?

A Local execution mode, running on a jump host inside the network

B Open inbound firewall rules for HCP Terraform's runners

C The API-driven workflow, which needs no network access

D An HCP Terraform agent running inside the network

◆ PAUSE AND DECIDE

What should they use?

A Local execution mode

Reaches the clusters and gives up all three things they asked for. Local mode disables policies, notifications and workspace variables.

B Open inbound rules

Removes a networking constraint on a private data centre, which is the opposite of what agents exist to avoid.

C API-driven

Confuses workflow with execution mode. An API-driven run still executes on HCP Terraform's infrastructure by default.

D An agent

Correct. The agent runs inside the network with an outbound connection, and runs stay HCP Terraform runs with everything intact.

ON THE EXAM

Domain 8. Local and Agent both move execution off HashiCorp's runners. Only the agent keeps the run under HCP Terraform's control.

> A global variable set defines an instance size for the whole organisation. A production workspace sets a workspace-specific variable with the same key and a larger size.

Which value does the production workspace use?

A The global variable set, because organisation scope wins

B The workspace-specific variable

C Neither. The run fails with a conflicting variable error

D Whichever was edited most recently

◆ PAUSE AND DECIDE

Which value does the production workspace use?

A The global set

Backwards. Organisation scope is the broadest scope, and breadth means lowest precedence unless the set is marked priority.

B The workspace variable

Correct. Workspace-specific variables always overwrite variables from variable sets with the same key.

C A conflict error

Overriding is the intended design. HCP Terraform shows which value it will use rather than refusing to run.

D Most recently edited

The common assumption, and wrong even for same-scope ties, which are broken by lexical order of the set names.

ON THE EXAM

Domain 8. More specific scope wins. The one exception is a priority variable set, which inverts the order deliberately.

Scoring paper 3

- **The answer key, in order:** D A B C A C B D A D C B D A C B A C D B.
- **Question 18 completed the fourth inverted pair**, against paper one's question 17 on hard-mandatory.
- **Across all three papers there were four inverted pairs.** If you missed both halves of one, revisit that module.
- **Sixty questions, weighted to this course's derived domain percentages**, verified across the whole set.
- Now read the trend, on the next slide.

> ON THE EXAM
>
> Every question across the three papers was original. None came from any question bank.

Reading the trend

- **Lay the three domain breakdowns side by side.**
- **A domain missed on all three is the only signal worth acting on.**
- **A rising score means little.** Later papers were not easier; you had seen more.
- **A falling score means little too.** Paper three carried the hardest misconceptions.
- **Where you missed matters more than how many.**

> ON THE EXAM
>
> The exam reports at objective level too. Practising that habit here means the real report tells you something you can use.

Before you sit the real thing

- **Revisit the modules your misses clustered in**, not the whole course.
- **Re-watch module twenty-four**, the confusion pairs, close to the day. It is pure recognition practice.
- **Check module twenty-five** if you use Terraform 1.15 or 1.16 at work. Three rules changed after 1.12.
- **Run the Certiverse system check days ahead**, and match your ID to your certification profile name.
- **Answer every question as you reach it.** A blank is a certain zero; a guess is not.

> ON THE EXAM
>
> Sixty minutes, one attempt, no free retake. Pace is part of the exam, and these three papers were where you practised it.